PA 1 Report

刘家豪 2024010779

`sort()` 函数说明

源代码如下.

```
void Worker::sort() {
    if (out_of_range) return; // if this process has no data, skip sorting

    // Calculate block size and active processes
    size_t std_block = ceiling(n, (size_t)nprocs);
    int activeprocs = (int)ceiling(n, std_block);

    // If only one process is active, just sort locally and return.
    if (activeprocs == 1) {
        radix_sort_float(data, block_len);
        return;
    }

    // Helper lambda to get the partner process for a given phase.
    auto get_partner = [this, activeprocs](int phase) {
        int p = (this->rank % 2 == phase % 2) ? this->rank + 1 : this->rank - 1;
        if (p < 0 || p >= activeprocs) return -1;
        return p;
    };

    // Helper lambda to get the actual block length for a given process rank.
    auto get_block_len = [&](int r) -> int {
        if (r < 0 || r >= activeprocs) return 0;
        size_t off = std_block * r;
        if (off >= n) return 0;
        return (int)std::min(std_block, n - off);
    };

    // local sort
    radix_sort_float(data, block_len);

    // Buffers for communication and merging
    float* swap_buf  = new float[block_len];
    float* recv_buf  = new float[std_block];
    float* cur  = data;
    float* aux  = swap_buf;

    MPI_Request reqs[2];
```

```
    for (int phase = 0; phase < activeprocs; ++phase) { // total activeprocs phases
        int p = get_partner(phase);
        if (p == -1) continue;

        int pcl = get_block_len(p);
        int tag = phase & 1;

        MPI_Irecv(recv_buf, pcl, MPI_FLOAT, p, tag, MPI_COMM_WORLD, &reqs[0]);
        MPI_Isend(cur, (int)block_len, MPI_FLOAT, p, tag, MPI_COMM_WORLD, &reqs[1]);
        MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);

        bool merged = false;

        if (rank < p) {
            // keep the lower block_len elements
            if (cur[block_len - 1] > recv_buf[0]) {
                int i = 0, j = 0, k = 0;
                while (k < (int)block_len) {
                    if (j >= pcl || cur[i] <= recv_buf[j]) aux[k++] = cur[i++];
                    else                                   aux[k++] = recv_buf[j++];
                }
                merged = true;
            }
        } else {
            // keep the upper block_len elements
            if (cur[0] < recv_buf[pcl - 1]) {
                int i = (int)block_len - 1, j = pcl - 1, k = (int)block_len - 1;
                while (k >= 0) {
                    if      (i < 0)                        aux[k--] = recv_buf[j--];
                    else if (j < 0 || cur[i] >= recv_buf[j]) aux[k--] = cur[i--];
                    else                                   aux[k--] = recv_buf[j--];
                }
                merged = true;
            }
        }

        if (merged) std::swap(cur, aux);
    }

    if (cur != data) // final copy back if the last merge result is in aux
```

```
        std::copy(cur, cur + block_len, data);

    delete[] swap_buf;
    delete[] recv_buf;
}
```

实现思路简述：先对每个进程本地数据进行排序（本实现使用基数排序，小规模时回退到 `std::sort`），再执行 odd-even transposition 的多轮邻居交换。每轮通信后，低 rank 保留较小一半数据，高 rank 保留较大一半数据，经过若干轮后达到全局有序。为减少开销，归并前先比较边界值，仅在确有交叉时才执行归并。

其中实现基数排序的辅助函数定义如下：

```cpp
// engineered to sort in ascending order by interpreting the bit pattern of the float.
static inline uint32_t float_to_sortable(float f) {
    uint32_t u;
    std::memcpy(&u, &f, sizeof(u));
    uint32_t mask = (u >> 31) ? 0xFFFFFFFFu : 0x80000000u;
    return u ^ mask;
}

static inline float sortable_to_float(uint32_t u) {
    uint32_t mask = (u & 0x80000000u) ? 0x80000000u : 0xFFFFFFFFu;
    u ^= mask;
    float f;
    std::memcpy(&f, &u, sizeof(f));
    return f;
}

static void radix_sort_float(float* arr, size_t n) {
    if (n <= 64) { std::sort(arr, arr + n); return; } // For small arrays, std::sort is faster t

    // Convert floats to sortable integers
    uint32_t* keys = new uint32_t[n];
    uint32_t* buf  = new uint32_t[n];

    // Transform the float array into a sortable integer array.
    for (size_t i = 0; i < n; i++)
        keys[i] = float_to_sortable(arr[i]);

    // Perform radix sort on the integer keys, processing 8 bits at a time.
    for (int shift = 0; shift < 32; shift += 8) {
        size_t cnt[256] = {};
        for (size_t i = 0; i < n; i++)
            cnt[(keys[i] >> shift) & 0xFF]++;

        size_t psum = 0;
        for (int b = 0; b < 256; b++) {
            size_t c = cnt[b];
            cnt[b] = psum;
            psum += c;
        }
```

```
        for (size_t i = 0; i < n; i++)
            buf[cnt[(keys[i] >> shift) & 0xFF]++] = keys[i];

        std::swap(keys, buf);
    }

    for (size_t i = 0; i < n; i++)
        arr[i] = sortable_to_float(keys[i]);

    delete[] keys;
    delete[] buf;
}
```

性能优化方式与结果

- 使用 `radix_sort_float` 进行本地排序，并在 `n <= 64` 时回退到 `std::sort`，兼顾大规模与小规模性能.
- 通过 `activeprocs` 仅让真实持有数据的进程参与 odd-even 迭代，减少空进程开销.
- 每一轮使用 `MPI_Irecv + MPI_Isend + MPI_Waitall` 完成邻居双向通信，保证交换逻辑清晰且正确.
- 归并前先做边界判断（如 `cur[block_len - 1] > recv_buf[0]`），无交叉时直接跳过归并，减少比较与写入.
- 通过 `cur/aux` 双缓冲加 `std::swap` 保存归并结果，避免每轮整段内存拷贝.

采取以上优化措施之后，使用提供的 `100000000.dat` 进行测试，性能对比如下.

进程数	优化前时间（ms）	优化后时间（ms）	优化后加速比
1×1	12275.305	3526.510	/
1×2	6758.324	2143.647	1.65x
1×4	3584.252	1338.911	2.63x
1×8	2020.664	913.647	3.86x
1×16	1274.377	732.856	4.81x

进程数	优化前时间（ms）	优化后时间（ms）	优化后加速比
2×16	1080.625	553.517	6.37x